Transforming non-linear problems

INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

datacamp

Maximize a non-linear function

- Artist produces up to 16 paintings at a cost $C(q) = \sqrt{q}$ where q is the quantity
- Inverse demand is $p = \frac{3}{\sqrt{q}}$ where p is the price
- $\max \Pi = pq - C = \frac{3}{\sqrt{q}}q - \sqrt{q} = 2\sqrt{q}$

$$\max 2\sqrt{q}$$

$$s.t. \quad q \leq 16$$



SciPy or PuLP?

- SciPy: `milp` expects a linear objective
- PuLP:

```
model = LpProblem('Artist', LpMaximize)
q = LpVariable('q', lowBound=0, upBound=16)
model += 2 * q**(1/2)
```

```
--> 3 model += 2 * q**(1/2)


TypeError: unsupported operand type(s) for ** or pow(): 'LpVariable' and 'float'
```

Substitute to linearize

- Substitute $z = \sqrt{q}$
- $\rightarrow \Pi = 2\sqrt{q} = 2z$
- $\rightarrow$ capacity constraint $z^2 \leq 16 \Leftrightarrow z \leq 4$

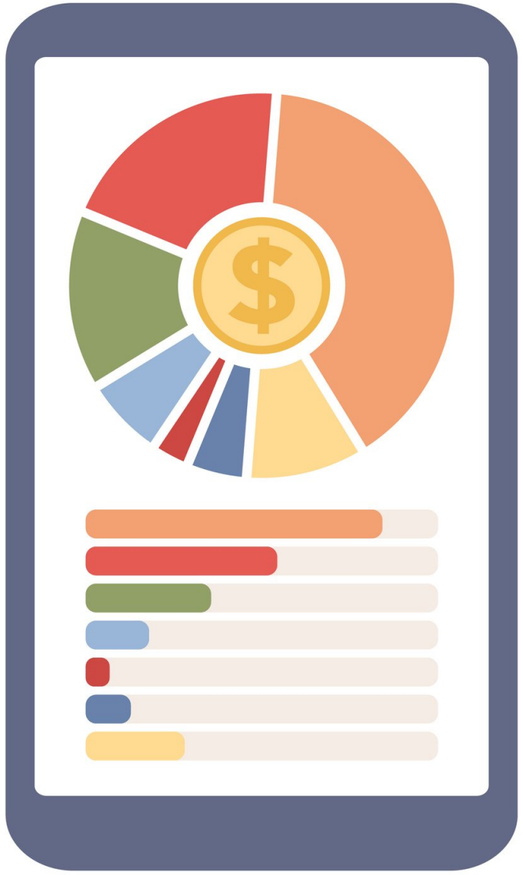
```
model = LpProblem('Artist', LpMaximize)
z = LpVariable('z', lowBound=0, upBound=4, cat='Integer')
model += 2 * z
model.solve()
print(f"Solution is {LpStatus[model.status]}.")
print(f"The optimal number of paintings is {round(z.varValue**2)}.")
```

```
Solution is Optimal.
The optimal number of paintings is 16.
```

Capital budgeting with dependent projects

Problem statement

- Projects A, B, C; A is a prerequisite for B
- Profits are respectively V = [250, 200, 300]
- Required investment is I = [2000, 1900, 2500] and only $4600 is available



Modeling

- o_A, o_B, o_C binary variables; indicate project selection

$$\max \quad o_A V_A + o_A o_B V_B + o_C V_C$$

$$s.t. \quad o_A I_A + o_A o_B I_B + o_C I_C \leq 4600$$

Linearization: product of binaries

- Replace $o_A o_B = o_{AB}$
- Add constraints
  - $o_{AB} \leq o_A$
  - $o_{AB} \leq o_B$
  - $o_{AB} \geq o_A + o_B - 1$
- Problem reduces to

$$\max \quad o_A V_A + o_{AB} V_B + o_C V_C$$

$$s.t. \quad o_A I_A + o_{AB} I_B + o_C I_C \leq 4600$$

$$o_A + o_B - 1 \leq o_{AB} \leq o_A, o_B$$

Resource allocation with training cost

- Allocate 120 tasks to minimize cost
- Senior (S), junior (J) and intern (I)
- Intern training cost $500
- Cost to resolve a task c = [30, 40, 5]
- Vector x: optimally assigned tasks
- Binary o: whether intern gets training
- $TC = 30x_S + 40x_J + (5x_I + 500)o$



Linearization: product of binary and continuous

- BigM method introduces a large number M
- Replaces the product with $z = (5x_I + 500)o$
- Imposes
  - $-oM \leq z \leq oM$
  - $-(1-o)M \leq z - (5x_I + 500)o \leq (1-o)M$

Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

Global optimization in SciPy

INTRODUCTION TO OPTIMIZATION IN PYTHON

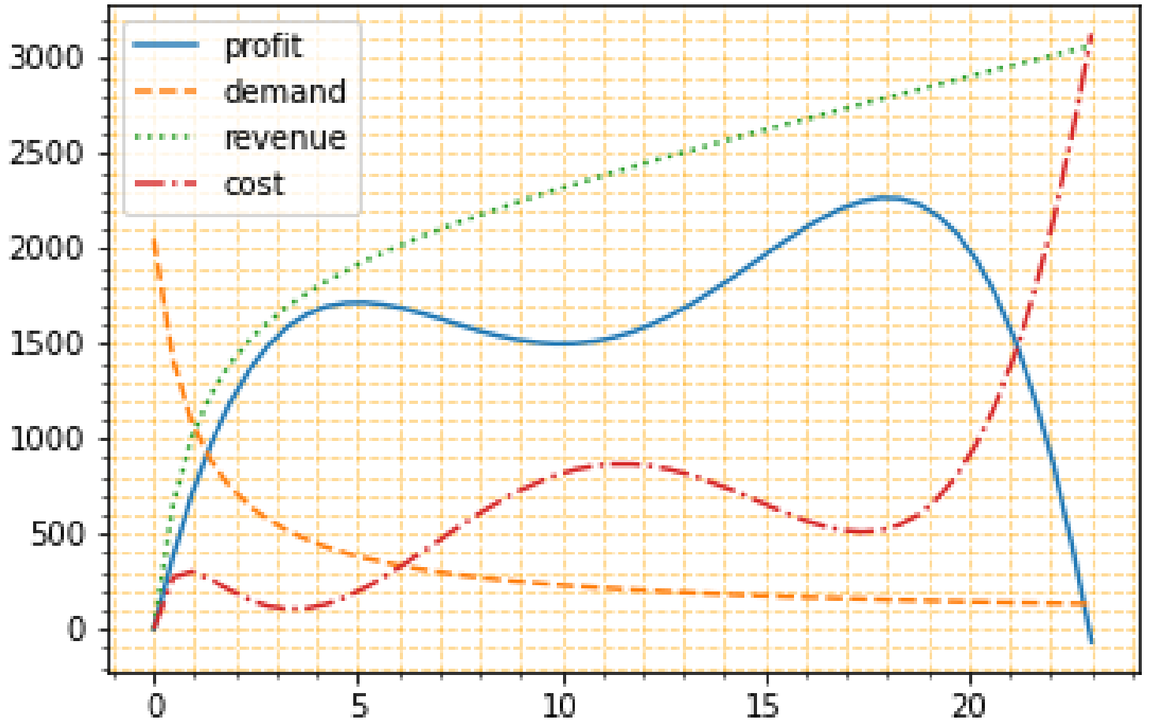


Jasmin Ludolf
Content Developer

datacamp

Global optimization

- Production alternates between being expensive and cheap
  - $\rightarrow \Pi(q)$ has two distinct maxima at 5 and 18. Minimum is at 10.



Getting stuck in a local optimum

```
from scipy.optimize import minimize_scalar, minimize

def profit(q):
  return -q**4 / 4 + 11 * q**3 - 160 * q**2 + 900 * q

result_min_scalar = minimize_scalar(lambda q: -profit(q))
print(f"{result_min_scalar.message}")
print(f"The maximum according to minimize_scalar is {-result_min_scalar.fun:.2f} \
and achieved at {result_min_scalar.x:.2f}\n")
```

```
Optimization terminated successfully;
The returned value satisfies the termination criteria
(using xtol = 1.48e-08 )
The maximum according to minimize_scalar is 1718.75 and achieved at 5.00
```

Getting stuck in a local optimum

```
x0 = 9
result = minimize(lambda q: -profit(q), x0, bounds=[(0, 20)])
print(f"{result.message}")
print(f"The maximum according to minimize(x0={x0}) is {-result.fun:.2f} at {result.x[0]:.2f}\n")
```

```
CONVERGENCE: REL_REDUCTION_OF_F_<=_FACTR*EPSMCH
The maximum according to minimize(x0=9) is 1718.75 at 5.00
```

```
x0 = 10 + 1e-8
result = minimize(lambda q: -profit(q), x0)
print(f"{result.message}")
print(f"The maximum according to minimize(x0={x0}) is {-result.fun:.2f} at {result.x[0]:.2f}\n")
```

```
Optimization terminated successfully.
The maximum according to minimize(x0=10.00000001) is 1500.00 at 10.00
```

Getting unstuck with basinhopping

```
from scipy.optimize import basinhopping


x0 = 0
result = basinhopping(lambda q: -profit(q), x0, niter=10000)


print(f"{result.message}")
print(f"The maximum according to basinhopping(x0={x0}, niter=10000) \
is {-result.fun:.2f} at {result.x[0]:.2f}\n")
```

```
['requested number of basinhopping iterations completed successfully']
The maximum according to basinhopping(x0=0, niter=10000) is 2268.00 at 18.00
```

basinhopping with kwargs and bounds

```
from scipy.optimize import basinhopping, NonlinearConstraint


x0 = 0
kwargs = {"constraints": NonlinearConstraint(lambda x: x, lb=0, ub=30)}


result = basinhopping(lambda q: -profit(q), x0, minimizer_kwargs=kwargs)
print(f"{result.message}")
print(f"The maximum according to basinhopping(x0={x0}) is at {result.x[0]:.2f}\n")
```

```
['requested number of basinhopping iterations completed successfully']
The maximum according to basinhopping(x0=0) is at 18.00
```

basinhopping with callback

```
from scipy.optimize import basinhopping, LinearConstraint

x0 = 0
kwargs = {"constraints": LinearConstraint([1], lb=0, ub=30)}

maxima = []

def callback(x, f, accept):
    if accept:
        maxima.append(*x)

result = basinhopping(lambda q: -profit(q),
                      x0,
                      callback=callback,
                      minimizer_kwargs=kwargs,
                      niter=5)
```

basinhopping - callback output

```
print(f"{result.message}")
print(f"The maximum according to basinhopping(x0={x0}) is at {result.x[0]:.2f}\n")
print(f"(Local) maxima found by basinhopping are: {[round(x, 2) for x in maxima]}")
```

```
['requested number of basinhopping iterations completed successfully']
The maximum according to basinhopping(x0=0) is at 18.00
(Local) maxima found by basinhopping are: [5.0, 5.0, 5.0, 5.0, 5.0, 18.0]
```

- Set `seed` argument to replicate results

Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

Sensitivity analysis in PuLP

INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

datacamp

Sensitivity analysis in linear programming

- **Sensitivity analysis** changes one parameter and to see changes in the output/outcome
- Linear programming: how much the objective changes
- Three approaches:
  1. Change a coefficient in the objective
  2. Change a coefficient in a constraint
  3. Change the constant term in a constraint
- Solve original and changed model
- Use PuLP!



Definitions

- **Constraint:**
  - Holds with equality: active, binding
  - Holds with strict inequality: non active, non-binding, slack (adj.)
- **Slack** (noun):
  - Amount to add to the left of constraint to make it binding
- **Shadow price:**
  - Marginal increase in the objective with respect to the right of the constraint

The diet problem in PuLP

Food	Cost ($/lb)	Protein (%)	Fat (%)
corn	0.11	10	2.5
soybean	0.28	40	1

- Recommendation: at least 17% protein, 2% fat, 7lb food
- Objective: $0.11C + 0.28S$
- Protein: $10C + 40S \geq 17(C + S)$
- Fat: $2.5C + S \geq 2(C + S)$
- Weight: $C + S \geq 7$

```
from pulp import *

model = LpProblem('diet', LpMinimize)

C = LpVariable('C', lowBound=0)
S = LpVariable('S', lowBound=0)

model += 0.11*C + 0.28*S

model += 10*C + 40*S >= 17 * (C+S), 'Protein'
model += 2.5*C + S >= 2 * (C+S), 'Fat'
model += C + S >= 7, 'Weight'

model.solve()
```

Slack and shadow price in PuLP

```
print(f"Status: {LpStatus[model.status]}\n")
```

```
for name, c in model.constraints.items():
    print(f"{name}: slack = {c.slack:.2f}, shadow price = {c.pi:.2f}")
```

```
Status: Optimal
```

```
Protein: slack = -0.00, shadow price = 0.01
Fat: slack = -1.05, shadow price = 0.00
Weight: slack = -0.00, shadow price = 0.15
```

Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

Congratulations!

INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

datacamp

Chapter 1

- Mathematical optimization
- Univariate optimization
- Multivariate optimization



Chapter 2

- Mathematical optimization
- Univariate optimization
- Multivariate optimization
- Unconstrained optimization
- Bounded and linear constraints
- Linear programming



Chapter 3

- Mathematical optimization
- Univariate optimization
- Multivariate optimization
- Unconstrained optimization
- Bounded and linear constraints
- Linear programming
- Convex constrained optimization
- Mixed integer linear programming



Chapter 4

- Mathematical optimization
- Univariate optimization
- Multivariate optimization
- Unconstrained optimization
- Bounded and linear constraints
- Linear programming
- Convex constrained optimization
- Mixed integer linear programming
- Transformed non-linear problems
- Global optimization and sensitivity analysis



The real-world



Congratulations!

INTRODUCTION TO OPTIMIZATION IN PYTHON